COMP7125 HKBU Study Assistant RAG Chatbot

1. ZHOU Tianchi Student ID: 25409638
2. ZHU Tianxiao Student ID: 25407732
3. LAN Ruipeng Student ID: 25463365
4. XIANG Bojie Student ID: 25400444

Abstract

This project presents a domain-specific Retrieval-Augmented Generation (RAG) chatbot designed to assist university students in accessing course information and generating personalized study plans. The system combines lexical and neural retrieval, query rewriting, and Chain-of-Thought (CoT) prompting, all integrated into an interactive graphical interface. Experiments evaluate retrieval performance, token efficiency, and the impact of each core component. The results show that neural embedding retrieval significantly outperforms keyword-based matching, query rewriting resolves multi-turn ambiguity, RAG eliminates hallucinations despite moderate token overhead, and the specialized CoT prompting proves essential for ensuring factually grounded responses by guiding the model through explicit verification steps.

1 Introduction

The key research problem of this work is: How to design a lightweight, locally deployed RAG chatbot that provides accurate course information retrieval and personalized study plan generation while reducing hallucinations and improving multi-turn dialogue understanding?

- It provides students with a reliable, context-aware academic assistant tailored to their actual course materials.
- It demonstrates how RAG can effectively improve the factual accuracy and reliability of LLM outputs in education scenarios.
- It explores practical techniques including query rewriting, dual retrieval, and Chain-of-Thought (CoT) prompting to enhance conversational quality.
- It delivers a user-friendly desktop GUI using CustomTkinter, supporting real-time streaming responses, visual CoT differentiation, dynamic feature toggles, token usage tracking, and conversation management to provide a stable and intuitive interaction experience for end users.

2 Implementation

This section describes the modular implementation of the system, focusing on core components, workflows, and prompt engineering design.

2.1 Module Overview and Implementation Checklist

The system follows a modular pipeline design, organized by functional blocks:

1. **Document Processing and Indexing** Documents (PDFs) are loaded using SimpleDirectoryReader and split into manageable chunks via SentenceSplitter. Each chunk is stored in a structured format, including the original file name and content, for later retrieval.
2. **Retrieval Module** Two retrieval methods are implemented: Lexical Retrieval matches keywords using Jaccard similarity between query and document chunks, while Neural Embedding Retrieval uses the embeddinggemma model to compute semantic similarity between query and document embeddings. The system returns the top 6 most relevant chunks based on the selected method, with configurable thresholds for filtering low-confidence results.

3. **Query Rewriting Module** Resolves ambiguous follow-up questions by incorporating conversation history. Converts context-dependent queries into standalone queries that can be accurately retrieved. Uses a lightweight LLM prompt to generate rewritten queries while preserving the original user intent.

4. **Prompt Assembly and Generation Module** Constructs the final prompt by combining the rewritten query, retrieved context, and conversation history. Supports type-specific prompt templates for both factual queries and study plan generation. Integrates **Chain-of-Thought** instructions to guide the model through structured reasoning. Interacts with the local Ollama `gemma3:4b` model for response generation. Supports streaming responses for real-time output. Controls generation parameters to ensure deterministic and focused outputs.

5. **Conversation Management** Maintains a stateful conversation history using a dedicated **ConversationManager** class. Formats the history into a string for prompt injection, enabling the LLM to maintain context across turns.

6. **Interactive GUI & Real-time Analytics** Built as a robust desktop application using the `customtkinter` framework. Features real-time streaming responses, visual differentiation of CoT (`<Thought>`) and final answers (`<Answer>`), and dynamic toggles for CoT and Query Rewriting. Provides real-time token usage tracking that accumulates costs from both the Query Rewriting and Final Generation steps.

7. **Evaluation** This implementation is backed by a comprehensive evaluation framework that quantifies the performance of each core component. Specifically, the system evaluates: (1) the precision of Lexical (Jaccard) vs. Neural (embedding) retrieval against ground-truth files; (2) the improvement in answer groundedness by comparing RAG and No-RAG responses; (3) detailed token metrics (Prompt, Completion, Total) for cost analysis; (4) the impact of query rewriting on retrieval precision in multi-turn conversations; and (5) the effect of the Chain-of-Thought (CoT) mechanism on response quality and hallucination control.

2.2 Key Prompt Design

The system's prompt engineering follows a structured three-stage pipeline to ensure high-quality, context-aware responses: (1) **Query Classification** to route the input, (2) **Dynamic Prompt Assembly** with query-type-specific templates, and (3) **Chain-of-Thought (CoT) Reasoning** to guide the LLM through a transparent, step-by-step thought process before generating the final answer. This design enforces factual grounding, prevents hallucinations, and produces well-organized outputs.

2.2.1 Query Type Classification

Before prompt generation, a lightweight rule-based classifier determines whether the query is `info` (factual/casual) or `plan` (study planning).

The classifier works in two stages:

1. **Keyword Matching**: It scores the query against predefined lists of keywords for each type.

2. **Pattern Boosting**: It applies regular expressions to detect structured patterns (e.g., "how to prepare", "study schedule"), which receive a score boost to prioritize `plan`-type classification.

This approach avoids an extra LLM call, ensuring fast and efficient routing.

2.2.2 Plan-Type Prompt

For study plan requests, the prompt guides the model to generate structured, actionable plans.

- The system prompt sets the role as an "HKBU study planner" and emphasizes realistic, cited plans.
- The CoT instructions follow a 6-step framework: Query Analysis, Constraint Extraction, Context Verification, Evidence Extraction, Gap Identification, and Plan Synthesis.
- The fixed markdown structure ensures the output is well-organized with sections for Overview, Weekly Schedule, Key Milestones, Resources, and Risk Mitigation.

2.2.3 Info-Type Prompt

For factual or casual queries, the prompt focuses on accuracy, transparency, and conversational flow.

- The prompt explicitly forbids fabricating information to prevent hallucinations.
- It handles both factual questions and casual chat in a single template.
- The CoT steps include "History Integration" to support multi-turn conversations and follow a 6-step logical deduction process for factual grounding.

Both prompt types enforce a strict XML-like response format (`<Thought>` and `<Answer>`) for consistent parsing and UI rendering.

```
# Query Type: plan
# System Message

    You are an HKBU study planner.
    Create realistic, actionable plans with clear timelines.
    Cite sources when using course info.
    If context is missing or not sufficient or not relevant, acknowledge it—never invent details.

# COT Instruction

        Follow this reasoning process for study plans:
        1. **Query Analysis**: Identify course/subject and planning needs
        2. **Constraint Extraction**: Extract time, goals, workload from query/history
        3. **Context Verification**: Check retrieved course information
        4. **Evidence Extraction**: Key facts from context (deadlines, requirements, topics)
        5. **Gap Identification**: What information is missing for a complete plan
        6. **Plan Synthesis**: Create structured, actionable study plan


# Retrieved Context
[No context available]
# Conversation History

# User Query
GIVE ME A STUDY PLAN
# Response Format

        Your response MUST follow this exact format:
            <Thought>:
                - Step 1 (Query Analysis): [Analysis]
                - Step 2 (Constraint Extraction): [Time, goals, workload]
                - Step 3 (Context Verification): [Context check]
                - Step 4 (Evidence Extraction): [Key facts]
                - Step 5 (Gap Identification): [Missing info]
                - Step 6 (Plan Synthesis): [Plan structure]
            </Thought>

            <Answer>:
                ## 📋 Study Plan Overview
                [Summary of recommendations]

                ## 🗓️ Weekly Schedule
                [Time-blocked schedule with specific tasks]

                ## 🎯 Key Milestones
                [Important deadlines and checkpoints]

                ## 📚 Resource Recommendations
                [Based on retrieved context]

                ## ⚠️ Risk Mitigation
                [Potential challenges and solutions]
            </Answer>

# Now generate your response
```

Figure 1: Prompt

3 Experiments and Results

3.1 RAG Retrieval Test: Lexical vs. Neural Retrieval

To evaluate the performance of the two retrieval methods, we designed a set of queries with known relevant documents. Precision was used as the evaluation metric, measuring the proportion of retrieved chunks that came from the correct document.

Table 1: Retrieval Quality Comparison: Lexical vs. Neural

Query	Ground Truth	Lexical Precision	Neural Precision
Introduction of COMP7125	COMP7125.pdf	33.33%	50.00%
Please introduce me Course COMP7125	COMP7125.pdf	0.00%	33.33%
COMP7125	COMP7125.pdf	66.67%	66.67%
Dr.Ma Shichao	Profile - Dr.MA.pdf	16.67%	66.67%
Introduce Dr.Ma's research interests	Profile - Dr. MA.pdf	16.67%	66.67%
Generate study plan for COMP7125	COMP7125.pdf	16.67%	66.67%

As shown in the results, the neural embedding retrieval outperformed the lexical method, achieving an average precision of 58.33% compared to 25.00%. This indicates that the neural method is more robust to phrasing variations and partial matches in the queries, leading to better document relevance on this dataset.

3.2 RAG vs. No-RAG Performance

Responses were compared for factual queries and study-plan queries.

Table 2: RAG vs No-RAG Token Usage Comparison

Metric	RAG	No-RAG	Difference
Prompt Tokens	1666	401	1265
Completion Tokens	633	654	-21
Total Tokens	2299	1055	+1244 (+117.9%)
Retrieved Chunks	6	0	-

As shown in Table 2, enabling RAG significantly increases token usage due to additional retrieved context. Although this introduces higher computational costs, it effectively eliminates hallucinations and ensures responses are grounded in accurate course information.

3.3 Query Rewriting Impact

This evaluates the impact of Query Rewriting on retrieval precision in multi-turn conversations, verifying the module's effectiveness in resolving contextual ambiguity and improving retrieval relevance.

Table 3: Effect of Query Rewriting on Retrieval Precision

Evaluation Index	Original Query	Rewritten Query	Improvement
Test Case 1 Precision	16.67%	83.33%	+66.67%
Test Case 2 Precision	0.00%	83.33%	+83.33%
Average Precision	8.33%	83.33%	+75.00%

Query rewriting significantly improved retrieval quality. The average precision increased by 66.67%. The rewriting step resolves ambiguities by incorporating conversation history, generating standalone queries that directly mention the target course or person.

3.4 CoT vs. No-CoT Impact

This experiment verifies the impact of Chain-of-Thought (CoT) prompting on response quality, hallucination control, and token consumption, validating the effectiveness of the scenario-specific CoT design in this system.

3.4.1 Key Output Comparison

- With CoT: The model explicitly identified no relevant information in the context and responded: "I'm sorry, but the provided documents do not contain any information about the assessment components for COMP7125." No false information was generated.
- Without CoT: The model skipped context verification, incorrectly applying assessment rules from other courses to COMP7125, inventing non-existent assessment weights and listing unrelated course details.

Table 4: Effect of Chain-of-Thought (CoT) Prompting

Evaluation Index	With CoT	Without CoT
Response Grounded in Context	Yes	No (hallucinated)
Token Usage	2440	2259
Token Difference		+359

3.4.2 Conclusion

Scenario-specific CoT prompting is essential to the system, improving both the interpretability and factual accuracy of responses, and aligning with the high-stakes needs of academic users.

4 Limitations and Future Work

- **Model Capacity Enhancement**: Optimize the model deployment scheme and adopt higher-performance open-source large language models to break through the limitations of local hardware, thereby significantly improving the system's reasoning ability, semantic understanding and response quality.
- **Query Classifier Upgrade**: Replace the original rule-based query classifier with an LLM-based intent classification model, which can accurately identify complex and ambiguous user queries and improve the accuracy of scenario adaptation.
- **Token Consumption Reduction**: Implement automatic summarization for retrieved context and multi-turn dialogue history. By compressing redundant information and retaining core content, the total token consumption can be effectively reduced without affecting the system's response performance.

5 Contribution Statement

- **XIANG Bojie**: Designed the system architecture, implemented the core RAG pipeline including dual retrieval, prompt engineering, query classification, and handled the majority of backend logic and debugging.
- **ZHOU Tianchi**: Developed the query rewriting module, integrated it with multi-turn conversation logic, and drafted the project report.
- **ZHU Tianxiao**: Conducted experiments and quantitative evaluation, generated result tables, and prepared the presentation slides.
- **LAN Ruipeng**: Built the CustomTkinter GUI, including streaming responses, CoT visualization, and token usage tracking.

6 Conclusion

This work developed a RAG-based academic chatbot tailored for HKBU course information queries and study plan generation. The system combined lexical and neural retrieval, dynamic prompt engineering with Chain-of-Thought reasoning, and a user-friendly GUI to address hallucinations and ambiguous multi-turn questions. Evaluation showed that neural embedding retrieval significantly outperformed lexical retrieval on the given dataset, while enabling RAG dramatically improved factual accuracy at the cost of higher token usage. The specialized Chain-of-Thought prompting proved essential for ensuring responses remained grounded in the provided context. Overall, the prototype demonstrates the practical value of lightweight, locally deployed RAG assistants for students, and provides a solid foundation for further optimization and extension.

7 References

[1] Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35, 24824–24837. https://arxiv.org/abs/2201.11903

[2] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. https://arxiv.org/abs/2005.11401

[3] LlamaIndex Documentation. https://docs.llamaindex.ai/

[4] Ollama Documentation. https://ollama.com/docs

[5] CustomTkinter Documentation. https://customtkinter.tomschimansky.com/documentation/